Multi-Agent Reinforcement Learning-Based Coordinated Dynamic Task Allocation for Heterogenous UAVs

Da Liu, Liqian Dou, Ruilong Zhang, Xiuyun Zhang, and Qun Zong

***Abstract*—The coordinated dynamic task allocation (CDTA) problem for heterogeneous unmanned aerial vehicles (UAVs) in the presence of environment uncertainty is studied in this paper. Dynamic task allocation mainly solves the problem of resource reallocation after new tasks appear, so that the multi-UAV systems can quickly respond to further information and objectives. In this paper, the CDTA strategy for heterogenous UAVs is proposed through proposer-responser mechanism and prioritized experience replay, in which the multi-agent reinforcement learning (MARL)-based coordinated network is constructed to propose request, and the Q-network is developed to approximate expected return to determine the responser whether to participate in the dynamic task. The CDTA algorithm considers the uncertainty of dynamic task and has a high scalability in different UAV groups, which can reduce the burden of online calculation and increase the speed of online operation effectively. The experiment proves that the priority experience replay speeds up the convergence of the algorithm, and the scalability of the algorithm is verified within 10-180 UAVs. Comparison simulations with the game theory-based and reinforcement learning-based methods are provided to show the effectiveness of the proposed algorithm.**

***Index Terms*—Coordinated dynamic task allocation, multi-agent reinforcement learning, heterogeneous unmanned aerial vehicles.**

I. Introduction

With the development of the UAVs' technology, UAVs in the large team becomes an important research, which are widely used in various industries such as disaster relief, unmanned combat, forest fire prevention and so on [1], [2], [3], [4]. Recent advances in intelligent multi-agent systems control promoted large group of UAVs acting cooperatively to accomplish tasks in dangerous and uncertain environments [5], [6], [7], [8], [9], [10], [11], [12], [13]. Particularly, the task allocation for multi-UAV plays a vital role in whether the task can be completed effectively. Assigning tasks to multi-UAV is a combinatorial optimization problem with several possible constraints [14]: heterogeneous UAVs may have different capabilities; different tasks performed by UAVs will have different benefits or costs, and new tasks may appear at any time; tasks may require coordination among several UAVs [15]. Therefore, under the above constraints, performing distributed autonomous task allocation fast and efficiently is quite challenging for the heterogeneous multi-UAV [16].

At present, there are mainly two methods of task allocation: centralized and distributed. Many genetic algorithms with different crossover policies are developed for this problem [17], [18], [19]. In the centralized setting, the communication, signal transmission, and control between UAVs in the system are all carried out by the single control center planner, which place the heavy processing requirement on the ground safely, thus making the UAVs smaller and cheaper to build. For the dynamic task allocation problem, an ant colony algorithm is developed by rerunning the static task allocation at a period time [20]. An extensive dynamic model is developed [21] for the stochastic nature of the cooperative search and task assignment problems, which can predict the value of unknown tasks and balance search and dynamic task response. In [22], an reinforcement learning-based method is designed to allocate heterogeneous UAVs for a sequence of tasks, but it can only allocate new UAV resources for the new tasks. In [23], [24], a genetic algorithm is adopted for multiple UAVs reallocation problems. A shared pool is designed to continuously generate possible solutions, from which a solution can be adapted when a new task comes. However, the centralized approach has a higher requirement for situational awareness. It is not feasible to adopt a purely centralized task allocation that needs all the knowledge of the UAVs and does not perform well in realtime computing when the number of the agents increases. And the centralized method results in the UAVs relying too much on the ground station and creating a single point of failure in the mission.

Compared with the centralized approaches, distributed algorithms can scale to larger team sizes and have advantages in realtime computing. In addition, the distributed task allocation method does not depend on the ground station for organization and coordination, making the multi-UAV system adaptable to different scenarios without the constraint of the communication

Manuscript received 16 August 2022; revised 15 November 2022; accepted 7 December 2022. Date of publication 12 December 2022; date of current version 18 April 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2018AAA0102401, in part by the National Natural Science Foundation of China under Grants 62003236, 62073234, and 62022060, and in part by the Science and Technology on Space Intelligent Control Laboratory, under Grant HTKJ2021KL502015. The review of this article was coordinated by Dr. Zehui Xiong. *(Corresponding author: Xiuyun Zhang.)*

Da Liu, Liqian Dou, Xiuyun Zhang, and Qun Zong are with the School of Electrical and Information Engineering, Tianjin University, Tianjin 300072, China (e-mail: liuda_2020@tju.edu.cn; douliqian@tju.edu.cn; zxy_11@tju.edu.cn; zongqun@tju.edu.cn).

Ruilong Zhang is with the Beijing Aerospace Automatic Control Institute, Beijing 100143, China (e-mail: 839267975@qq.com).

Digital Object Identifier 10.1109/TVT.2022.3228198

0018-9545 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

infrastructure [25]. For example, when the multi-UAV system performs a task, each UAV can communicate with their peers and can make a decision based on their own observation without the central planner, which can make the system more robustness in case of the failure of the central planner [26]. What's more, in the distributed task allocation, the system requires UAVs to have independent calculation, analysis and decision-making capabilities. For the auction-based distributed planner in task allocation, Ref. [27] introduces the consensus-based bundle algorithm (CBBA), which can provide an approximate solution to the vehicle routing problem when all the tasks are introduced at the beginning of the algorithm. However, in dealing with the dynamic task allocation, the full resolving of CBBA ignores the fact that all of the UAVs has already arrived at a conflict-free status, wasting computation and communication resources for the original task allocation. Ref. [28] introduces the CBBA with partial replanning algorithm, which extends the work in [27] to adapt to new tasks by resetting a portion of UAVs' previous allocation. Ref. [29] introduces PI-MaxAss algorithm which is efficient for reassignment of allocated tasks by taking advantage of existing schedule space. Ref. [30] proposes an agent-based task allocation mechanism based on the auction process, which is effective for assigning dynamic tasks and achieve high performance of the UAV swarm. Ref. [31] integrates the market approach and the modified contract network protocol to assign the tasks for heterogeneous robots, which enhances the overall efficiency of task allocation by setting trust, capability and other parameters during the auction. For the game-based dynamic task allocation, Ref. [32] designs a potential game-theoretic task allocation framework to help each agent make a reasonable choice in response to the unexpected environment change and adjust the assignment to complete tasks simultaneously. Ref. [33] proposes a game-theoretical autonomous decision-making framework to address a task allocation problem for a swarm of multiple agents, in which a portion of UAVs can adapt to the new task in the method. In general,the auction-based methods are more suitable for dynamic tasks and the game-based approaches are suitable for large-scale task allocation, but all require global communication in the group, which creates a burden for the large number of UAVs. As an intelligent decision-making method, reinforcement learning(RL) provides long-term robust strategies for agents in uncertain environments [34]. In [35], an decentralized multi-UAV Q-learning algorithm is introduced to design the UAV trajectory considering the sensing and transmission processes in the dynamic environment. Further, in [36] an enhanced multi-UAV Q-learning algorithm is introduced to solve the dynamic trajectory control problem of the UAVs with the power management and subchannel allocation. In [37], CA2C a combined reinforcement learning-based method is proposed to design the trajectories of multiple UAVs and task selection strategy in the cooperative sensing tasks with the objective of minimizing the accumulated age of information (AoI) of the static targets. Recent results of RL research provide a way to solve complex tasks, and the trained networks can even surpass the level of human experts in complicated scenarios [38], [39]. Reinforcement learning also shows promising results in the field of multiple agents control [40], [41]. Besides, its computation time is similar to simple heuristic methods after training. The above shows that dynamic task allocation based on RL is a problem worth studying.

To sum up, the major difficulties in dynamic allocation problems are: (1) Ignoring the initial conflict-free status of the UAVs before reallocation, which contributes to the waste of computation and communication resources. (2) The auction-based and game-based dynamic allocation methods still requires all UAVs to communicate multiple times, which increases the burden of the system. Therefore, to solve the above problems and better trade-off efficient use of resources with the speed of making decisions, in this paper, a bidirectional request-response dynamic task allocation algorithm is developed based on deep reinforcement learning, enabling the UAVs to reallocate a portion of their existing allocation with partial UAVs communication. The simulations show that the priority experience replay speeds up the convergence of the algorithm and the scalability of the algorithm within 10-180 UAVs. Furthermore, comparative experiments with the game theory-based [33] and reinforcement learning-based [22] methods are provided to show the effectiveness of the proposed algorithm. The contributions of this paper are as follows.

- The Coordinated Dynamic Task Allocation (CDTA) is proposed to allocate the dynamic tasks with the existing resources of the UAVs by designing a bidirectional request-response mechanism. The designed approach can achieve the desired fast response performance without global communication, which reduces the online calculation and improves the robustness in communication-restricted environments.
- A multi-agent reinforcement learning (MARL)-based coordinated request network is developed to request other UAVs directionally, which has a high scalability in the quantity and types of UAVs. Prioritized experience replay is combined with the MARL, which improves the convergence speed of the algorithm.
- A Q-learning-based response network is developed to approximate the expected return of the UAV participating in the dynamic task, maximizing the utility after reallocating the new task to the team of the UAVs. Parameter sharing method is adopted to reduce the parameter space and accelerate the learning speed.

The remainder of this paper is organized as follows. In Section II, the dynamic task allocation problem formulation and the parameter definition are presented. In Section III, the dynamic task allocation problems are modeled as MDPs. The details of the reinforcement learning algorithm for the UAVs dynamic task allocation are explained. Then in the Section IV, the distributed dynamic task allocation strategy is presented. The effectiveness of the proposed method is verified in Section V by numerical simulation. The conclusion is given in Section VI.

II. Problem Formulation

This section presents the dynamic task reallocation scenario of heterogeneous UAVs that execute specific task on stationary targets. Then the relevant parameters of UAV and task are defined. Finally, the mathematical formulation is used to describe the process of task allocation.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

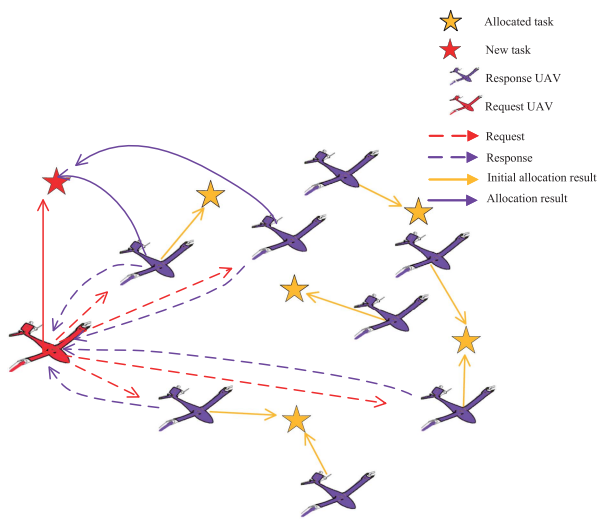

Fig. 1. Dynamic task reallocation scenario description.

A. Scenario Description

Suppose a set of UAVs performing specific tasks on stationary targets, such as attack or reconnaissance tasks in the urban combat. Each task requires UAVs with the corresponding abilities and can be executed independently of other tasks. Now assume that a new task appears after the initial task allocation of the UAVs, and a set of the UAVs must be reallocated, which can be shown more clearly in Fig. 1. In addition, the resources of the UAV are redundant for each task in the initial allocation, which means that even after the reallocation of the new task, the UAV's resources can meet the needs of all tasks. The scenario is mainly divided into the following parts:

- $Propose$: In the distributed UAV combat, the request UAV that discovers a new task determines the response UAV according to the information of the new task, then sends the task information to the response UAV, as shown by the red dashed line.
- $Response$: The response UAVs make their own corresponding evaluations based on the importance, urgency of the new task and the distance from the task, then send the evaluation results and resource information to the request UAV, as shown by the purple dashed line.
- $Decision$: Based on the received information, the request UAV decides whether the response UAV will participate in the new task. The allocation will be terminated until new tasks no longer generate.

Therefore, this article introduces a MARL algorithm to determines the response UAVs in $Propose$, and a Q-learning-based neural network to evaluate the new task in $Response$. The UAVs participating in the new task follow the principle of the shortest distance in $Decision$.

B. Parameter Definitions

(1) Tasks: Let $Task_UAV = \{T_1, T_2, \ldots, T_j\}$ be the set of the initial task, and T_j represents the task being performed by UAV_j, $Require_T_k = \{att(T_k), rec(T_k), jam(T_k), com(T_k), bom(T_k)\}$ denotes the task T_k's requirement of attack capability, reconnaissance capability, jamming capability, communication and bombing capability. $Task_new$ means the new task encountered by UAVs. Noted that the environment of the battlefield is complex and uncertain, so the importance, urgency, distance, and requirement of the new task are randomly initialized. (2) UAVs: Let $\{U_1^1, U_2^2, \ldots U_M^j\}$ be the set of M heterogeneous UAVs, where the superscripts j denotes the type of the UAV and the subscript M represents the ID of the UAV. Let $Capability_U^j\{att(U^j), rec(U^j), jam(U^j), com(U^j), bom(U^j)\}$ denotes the corresponding capability of the type of UAV. UAVs with different capabilities constitute heterogeneous diversity and also increase the complexity of task assignment. Let $Responser = \{U_1^r, U_2^r, \ldots, U_n^r\}$ denotes the set of response UAVs and $Proposer$ denotes the request UAV. Let $ResponserMessage = \{A_1, A_2, \ldots, A_n\}$ denotes the message from the responsers. $Re_allcation = \{u_1, u_2, \ldots, u_k\}$ represents the UAVs participating in the new task.

C. Mathematical Formulation

1) Constraints: In addition to meeting existing tasks'requirement, the UAVs participating in the new task should meet the following boundary conditions.

$$\sum_{i=1}^{M} att(u_i^j) > att(T_{new}),$$
$$\sum_{i=1}^{M} rec(u_i^j) > rec(T_{new}),$$
$$\sum_{i=1}^{M} jam(u_i^j) > jam(T_{new}),$$
$$\sum_{i=1}^{M} com(u_i^j) > com(T_{new}),$$
$$\sum_{i=1}^{M} bom(u_i^j) > bom(T_{new}), \tag{1}$$
$$\min \sum_{i=1}^{M} dis(u_i^j), \tag{2}$$

where M means the number of selected UAVs which will participate in the new task and dis refers to the distance from the selected UAV to the target. (1) means that the total combat ability of the selected UAVs should be greater than the ability required to complete the task. (2) means that the UAV can reach the target location in the shortest time. (1) and (2) guarantee that the selected UAVs for the new task can meet the requirement of safety and efficiency simultaneously.

2) Evaluation function: The resource of the UAV is limited, so it's essential to control the resource consumption in the allowable range for the task. Hence the following

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

evaluation function is established.

$$J = \left(\left(\sum_{i=1}^{M} att(u_i^j) - att(T_{new}) \right)^2 + \left(\sum_{i=1}^{M} rec(u_i^j) - rec(T_{new}) \right)^2 + \left(\sum_{i=1}^{M} jam(u_i^j) - jam(T_{new}) \right)^2 + \left(\sum_{i=1}^{M} com(u_i^j) - com(T_{new}) \right)^2 + \left(\sum_{i=1}^{M} bom(u_i^j) - bom(T_{new}) \right)^2 \right)^{\frac{1}{2}} \leq Th, \tag{3}$$

where Th is a user-defined task completion threshold. Eq. (3) illustrates the difference between the new task required capability and the selected UAVs capability must be limited to a specific range.

The paper aims to propose a new method for dynamic task reallocation in the distributed multi-UAV system with partial communication and existing UAV resource under the evaluation function (3) with (1)–(2).

III. Cooperative Deep Reinforcement Learning-Based Task Allocation Strategy

In this section, the $Propose$ and $Response$ in Section II will be modeled as a Markov decision process, and reinforcement learning-based algorithm is developed to solve the dynamic task allocation problem.

A. Markov Decision Process

A Markov decision process (MDP) is a stochastic process that satisfies the Markov property. The element can be represented by tuples $< S, A, R, P_{ss}, \gamma >$. S represents the state space; A represents the action space; R represents the immediate reward obtained after acting action $a \in A$. $P^a_{SS'}$ defines the probability of observing state $s' \in S$ after executing an action $a \in A$ from state $s \in S$. γ defines the discount factor. The MDP problem is to find a policy to maximize the cumulative reward, which can map the optimal action a in the state s.

$$\max_{\pi \in \prod} E\left[\sum_{n=1}^{N} \gamma^{n-1} R_{n-1} \,|\pi \right]. \tag{4}$$

For dynamic task that arrives randomly, it is assumed that $Task_new(t)$ represents the dynamic task that arrives at time t. Due to the uncertainty of the battlefield environment, the dynamic task that arrives at time $t + \Delta t$ has no related with the allocated tasks $Task_UAV = \{T_1, T_2, \ldots, T_j\}$ that arrive before time t, and specifically, the requirement of the $Task_new(t)$ is also independent of the allocated tasks $Task_UAV = \{T_1, T_2, \ldots, T_j\}$. Therefore, the dynamic task allocation for $Task_new(t, t > 0)$ satisfies the Markov property. In particular, the Markov decision processes for the request and response of dynamic task allocation are established respectively.

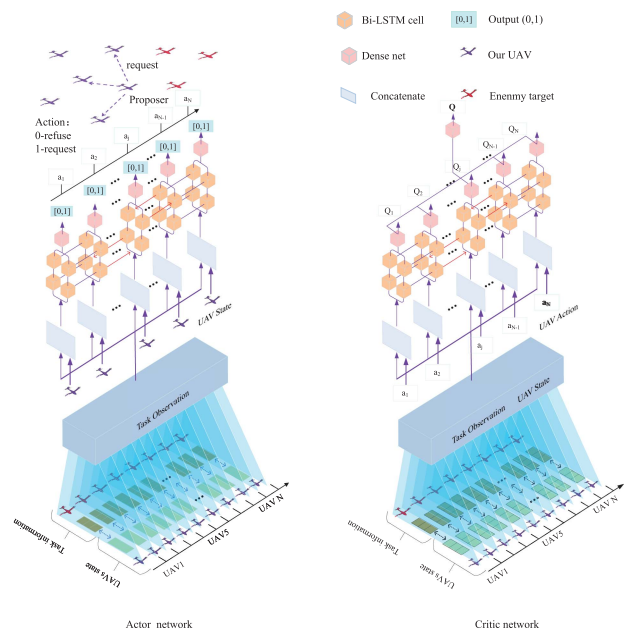

Fig. 2. Coordinated Dynamic Task Allocation Request Net. As a means of communication, bi-direction Long Short-Term Memory(LSTM) networks are used to connect each UAV's policy and Q networks.

B. MDP for the Request UAV

The $Proposer$'s CDTA request net receives task information, the state of each UAV, and outputs the optimal $Responsers$. The above process is modeled as a Markov decision process. The components are as follows:

State space:

$$s_p = \{Require_Task_new, Task_UAV\} \in S_p. \tag{5}$$

The initial state of the environment is randomly generated from combat scenarios.

Action space: The selected UAVs' ID ready to send request is defined as action a. So the action space is defined as the set of the UAVs:

$$A_p = \{U_1, U_2, \ldots, U_M\}. \tag{6}$$

Reward: Taking into account the constrains (1) and evaluation function (3), the following reward function is designed:

$$R_p = \begin{cases} r_p - \rho \sum dis(U_i^r, T_{new}) & \text{if satisfying Eq.(1),Eq.(3)} \\ -r_p & \text{else,} \end{cases} \tag{7}$$

where r_p is the expert experience value, ρ denotes the distance coefficient and the $\sum dis(U_i^r, T_{new})$ means the sum of the distance for the $Responsers$ to the new task.

Inspired by BiCNet [42], the bidirectional recurrent network is adopted and the model is designed under the actor-critic framework. The actor network and critic network of the Coordinated Dynamic Task Allocation Request Net are illustrated in Fig. 2.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

The bidirectional recurrent network structure can provide a communication channel for multi-agent communication and serve as a memory storage unit, which can be trained on a small team, and scale up to more UAVs, dealing with different combat scenarios. Each agent can maintain its own state and share information with collaborators. Due to the structural characteristics of the LSTM network, the parameters are shared between different nodes, which can not only accelerate the learning speed of the model, but also improve the domain adaption ability.

When calculating and determining the response UAVs, the problem of resource wasting in (3) is not considered. We only need to guarantee that the optimal result is in the set of the response UAVs. With the defined reward, the $Proposer$ takes action, gets an immediate reward r and aims to learn a policy that can maximize the sum of discounted reward in (4). Consequently, the following Bellman Optimal Equation is introduced:

$$Q^*(s_p, a) = r(s_p, a) + \gamma Q^*(s'_p, a'), \tag{8}$$

where $Q^*(s, a)$ is the optimal action-state value function. Here the deterministic policy gradient is used for multi-agent learning. The temporal difference (TD) method is adopted to update the neural network, which can learn from a part of the raw experience without the whole sequence and the model of the environment. Besides, a dual network structure is adopted to improve the stability of the algorithm convergence. The loss function of the critic network is as follows:

$$\begin{aligned} loss &= E_{s_p,a,r,s'_p}[(Q_i^{\pi}(s_p, a_1, \ldots, a_M) - y)^2], \\ y &= r + \gamma Q_i^{\pi'}(s_p, a'_1, .., a'_M)\,|_{a'_j = \pi'_j(s_p)}\ . \end{aligned} \tag{9}$$

In (9), the sum of square loss is used to compute the function. The trained critic network is used to estimate the policy value. In training the policy network: given UAVs which are collectively represented in a policy parameterized with θ, the following policy gradient is as follows:

$$\begin{aligned} &\nabla_\theta J(\pi) = E_{s_p, a\sim D}\left[\nabla_\theta \pi_i\left(a_i \mid s_p\right)\right. \\ &\left.\nabla_{\pi_i} Q_i^{\pi}\left(s_p a_1, \ldots, a_M \mid a_i = \pi_i\left(s_p\right)\right)\right] \end{aligned} \tag{10}$$

where D is a replay buffer, including the historical information of the UAVs. The prioritized experience replay (PER) is adopted in training agents.

Uniform sampling from the buffer is a classic method in reinforcement learning. However, different experiences have different effects on the training model. Some experience is of ' higher value' and can have significant influence on the current training, while others can not. Schaul et al. [43] proposed a framework for prioritizing experience, replaying important transitions more frequently, and learning more efficiently for single agent reinforcement. Here we extend this method to the field of multi-agent reinforcement learning. Similar to Schaul, the temporal-difference error can be measured as the importance of the priority. In single agent leaning, the probability of sampling the experience transition is calculated as follows:

$$\begin{aligned} P(i) &= \frac{p_i^{\alpha}}{\sum_{k=1}^{K_b} p_k^{\alpha}}, \\ p_i &= |TD(i)| + \varepsilon, \end{aligned} \tag{11}$$

where p_i is the priority of experience transition i, K_b refers to the size of the buffer, the exponent α determines how much prioritization is used, with $\alpha = 0$ corresponding to the uniform sampling. ε is a small positive constant that prevents the transitions not being revisited once the TD error is zero. In the multi-agent case, considering the Q value of M agents, it is beneficial to take advantage of the sum of the TD-error of all agents as the priority. The probability of the experience transition can be calculated as follows:

$$p_i = \sum_{j=1}^{M} |TD_i(j)| + \varepsilon, \tag{12}$$

When using the experience transition data to update the weight of the critic and actor network in (9) and (10), the loss function of the neural network is computed by the replay experience data, which is obtained through sampling in the uniform distribution from the experience buffer organized in a queue data structure. Therefore, the accuracy of the estimation for the expected Q value relies on the experience data corresponding to the real distribution in the experience buffer. Since the goal of the critic network is to minimize the TD errors, we calculate the proportional sampling probability of each sample based on the TD errors, and give priority to the samples with larger TD errors to update the network weights, which changes the original distribution of the experience samples. However, the sample distribution in the non-real situation will directly lead to the inaccurate estimation of Q(s, a) by the critic network, and lead to the wrong direction of the gradient update of the actor network. Therefore, the sample distribution bias caused by the prioritized replay of experience will lead to the change of the solution that the estimates will converge to. The bias can be corrected by using importance sampling (IS) weight:

$$w_i = (\tfrac{1}{M} \cdot \tfrac{1}{P(i)})^{\beta}, \tag{13}$$

where the bias can be fully compensated for prioritized experience replay with $\beta = 1$. Then the (9) is recalculated as follows:

$$\begin{aligned} &loss = E_{s_p,a,r,s'_p}[w_i \cdot (Q_i^{\pi}(s_p, a_1, \ldots, a_M) - y)^2], \\ &y = r + \gamma Q_i^{\pi'}(s_p, a'_1, .., a'_M)\,|_{a'_j = \pi'_j(s_p)} \end{aligned} \tag{14}$$

Adaptive moment estimation (Adam) method is adopted to optimize (10) and (14).

With the components of MDP defined above, it can be concluded that the actor and critic network are trained to obtain the optimal $Responsers$ for the $Proposer$.

C. MDP for Response UAV

As shown in Fig. 3, each response UAV uses the Deep Q-Learning (DQN) method to calculate the expected utility of the current state and the new task, then decide whether to accept the request from the $Proposer$. The MDP is modeled as follows:

State space:

$$\begin{aligned} &s_r = \{Require_Task_new, Rwequire_T_j, \\ &importance, fuel_cost\} \in S_r \end{aligned} \tag{15}$$

where the state space includes the requirement of the task being performed by the response UAV and the new task, also consists of the importance, and fuel cost of the new task.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

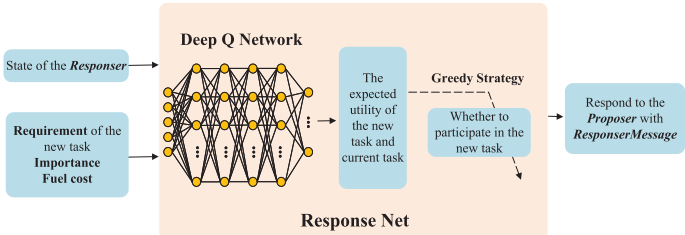

Fig. 3. Deep Q Learning-based Response Net.

Action space: The response UAV decides whether to participate in the new task.

$$A_r = \{True, False\}, \tag{16}$$

where $True$ means that the response UAV decides to participate in the new task, and $False$ is the opposite.

Reward: The UAVs who choose task with higher utility will be given positive reward.

$$R_r = \begin{cases} r_r - dis(U_i^r, T_{new}) & \text{if satisfying Eq.(1) and Eq.(3)} \\ -1 & \text{else,} \end{cases}$$
$$r_r = \delta_k(Imp_{new} - Imp_{old}), \tag{17}$$

where r_r denotes the response UAV will be rewarded when they choose the more important tasks, δ_k means the task importance factor, and $dis(U_i^r, T_{new})$ denotes the distance from the response UAV to the new task.

In implementing of DQN for multi-UAV task allocation, the learning process is accelerated by sharing network parameters among UAVs. In the distributed execution, the UAV evaluates the reward of the new task through its neural network and responds to the $Proposer$. Although the UAVs learn from one network and the parameters of the decision-making network are the same, they can behave differently because of the different state of each UAV. To update the network parameters efficiently, the gradient descent method is used to train the network and use the dual network structure to improve the convergence performance of the algorithm. The gradient descent learning for loss function is demonstrated as follows:

$$loss = E[(r + \gamma \max_{a'} Q(s_r', a_r') - Q(s_r, a_r))^2]. \tag{18}$$

A challenging problem in reinforcement learning is how to weigh the relationship between exploration and exploitation. If the UAVs always choose the best action, they are likely to fall into a local optimum. On the other hand, if the UAVs are more inclined to explore, the algorithm's convergence may also become a problem. In the experiment, the $\varepsilon - greedy$ strategy is used, which selects the optimal action with the probability $1 - \varepsilon$, and randomly generates the action with the probability ε for exploration.

D. Algorithm Analysis

In this subsection, the computational complexity and the convergence of the proposed CDTA algorithm are analyzed.

1) Complexity of Action Selection: First, the Proposer sends directional requests to the Responsers based on (5), which are the part of the M UAVs, so the computational complexity in the process of Propose is $O(M)$. Then, the Responsers, responds to the request based on the deep Q network respectively according to (15), so the computational complexity of the Response is $O(1)$. In the end, the Proposer determines the UAVs sequence for task reallocation according to the task requirement considering the distance between the UAV and the task target in (1) and (2). In the calculation, it is necessary to sort the distance of the UAV and judge whether it meets the task requirements, so the computational complexity is $O(M \log(M))$. Therefore, the overall computational complexity of the proposed algorithm is $O(M \log(M))$.

2) Complexity of Training Process: Given the batch size of training samples as N_b and the experience buffer size as K_b. Before training, the critic network and Q network need to calculate the Q value for the Proposer and Responsers. The computational complexity of the LSTM is linear with the input sequence, both at training and inference in [44]. Therefore, according to (14) and (18), the computational complexity of the Q calculation for the request network and response network are $O(M \cdot N_b)$ and $O(N_b)$. Moreover, for a fixed network with constant layers and neurons, the computational complexity of the network is linear with the input and the output [45]. Therefore, the computational complexity of the back-propagation for request network and response network are $O(M^2 \cdot N_b)$ and $O(N_b)$ respectively. Besides, for the Request UAV, the complexity of sampling and updating of the prioritized experience replay are $O(1) + O(\log(K_b))$. In summary, the computational complexity of the training is $O(M^2 \cdot N_b)$.

3) Convergence: The key point for the convergence of the Propose and Response network is whether their critic network (Q network) could approximate the optimal action-value function, denote as $Q^*(s, a)$. Therefore, the convergence of the critic network is proved as follows. The update rule of the critic network (Q network) as shown in (9) and (18) could be rewritten as follows:

$$Q_{k+1}(s_k, a_k) = Q_k(s_k, a_k) + \alpha_k(s_k, a_k) \times [r_k + \gamma \max Q_k(s_{k+1}, a_{k+1}) - Q_k(s_k, a_k)], \tag{19}$$

where the $\alpha_k(x_k, a_k) \in (0, 1)$ represents the step size, and for more brevity the $Q(s_p, a_1, \ldots a_M)$ and $Q(s_r, a_r)$ are denotes as $Q_k(s_k, a_k)$ at the kth update.

Theorem 1: Given a finite $MDP(S, A, R, P, \gamma)$, the critic network, which updates with the rule in (19) converge to the optimal critic network $Q^*(s, a)$ w.p.1 as long as

$$\sum_t \alpha_t(s, a) = \infty$$
$$\sum_t \alpha_t^2(s, a) < \infty. \tag{20}$$

It should be noted that, $\alpha_t(s, a) \in (0, 1)$, and (20) requires that all of the state-action pairs should be visited infinitely when updating the network. To prove the correctness of the Theorem1, an auxiliary results of a Stochastic Process is shown as follows.

Lemma 1 [46]: The random process $\{\Delta_k\}$ taking values in R^n and defined as

$$\Delta_{k+1}(s) = (1 - \alpha_k(s))\Delta_k(s) + \alpha_k(s)F_t(s), \tag{21}$$

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

converges to zero w.p.1 with the following conditions:

$$\begin{aligned} &0 \le \alpha_t \le 1, \sum_t \alpha_t(x) = \infty \text{ and } \sum_t \alpha_t^2(x) < \infty \\ &\|E[F_t(x) \mid \mathcal{F}_t]\|_W \le \gamma \|\Delta_t\|_W, \text{ with } \gamma < 1 \\ &\text{var}\,[F_t(x) \mid \mathcal{F}_t] \le C\left(1 + \|\Delta_t\|_W^2\right), \text{ for } C > 0. \end{aligned} \tag{22}$$

Proof of Theorem 1: First, (19) could be rewritten as follows:

$$\begin{aligned} Q_{k+1}(s_k, a_k) &= (1 - \alpha_k(s_k, a_k))Q_k(s_k, a_k) \\ &\quad + \alpha_k(s_k, a_k)[r_k + \gamma \max Q_k(s_{k+1}, a_{k+1})]. \end{aligned} \tag{23}$$

Subtract the optimal action-value function $Q^*(s_k, a_k)$ and let

$$\Delta_k(s) = Q_k(s, a) - Q^*(s, a). \tag{24}$$

Then, (23) could be written as

$$\begin{aligned} \Delta_{k+1}(s) &= (1 - \alpha_k(s_k, a_k))\Delta_k(s) + \alpha_k(s_k, a_k)[r_k \\ &\quad + \gamma \max Q_k(s_{k+1}, a_{k+1}) - Q^*(s, a)]. \end{aligned} \tag{25}$$

Now let

$$\mathrm{F_k(s, a) = r_k + \gamma max Q_k(s_{k+1}, a_{k+1}) - Q^*(s, a)}. \tag{26}$$

Next, we will analyze whether $F_k(s, a)$ satisfies the conditions of (22), and let H denotes the contraction operator. For a generic function $Q(s, a)$, it follows

$$(HQ)(s, a) = \sum_{s \in S} P_{ss'}[r(s, a) + \gamma \max Q(s', a')], \tag{27}$$

and according to the principle of contraction, we have

$$\|HQ_1 - HQ_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty. \tag{28}$$

The $\|E[F_t(x) \mid \mathcal{F}_t]\|$ could be expressed as follows,

$$\begin{aligned} \|E[F_t(x) \mid \mathcal{F}_t]\| &= \sum_{s \in S} P_{s_k s_{k+1}}[r_k + \gamma \max Q(s_{k+1}, a_{k+1}) \\ &\quad - Q^*(s, a)] \\ &= (HQ_k)(s, a) - Q^*(s, a), \end{aligned} \tag{29}$$

where the fact is that $HQ^* = Q^*$ and Q^* is a fix point of contraction operator H.

Then,

$$\|E[F_t(x) \mid \mathcal{F}_t]\| = (HQ_k)(s, a) - (HQ^*)(s, a). \tag{30}$$

According to (28), it is obvious that

$$\|E[F_t(x) \mid \mathcal{F}_t]\|_\infty \le \gamma \|Q_k - Q^*\|_\infty = \gamma \|\Delta_k\|_\infty. \tag{31}$$

Moreover, $\text{var}[F_k(s) \mid \mathcal{F}_k]$ could be expressed as

$$\begin{aligned} \text{var}\,[F_k(s) \mid \mathcal{F}_k] &= [(r_k + \gamma \max Q(s_{k+1}, a_{k+1}) - Q^*(s, a) \\ &\quad - (HQ_k)(s, a) + Q^*(s, a))^2] \\ &= \text{var}(r_k + \gamma \max Q(s_{k+1}, a_{k+1}) \,|\mathcal{F}_k), \end{aligned} \tag{32}$$

where r_k is a bounded value and it is clearly verifies that

$$\text{var}\,[F_k(s) \mid \mathcal{F}_k] \le C(1 + \|\Delta_k\|_W^2), \tag{33}$$

with the constant value C.

Finally, by Lemma 1, Δ_k converges to zero w.p.1, and the action-value function $Q(s, a)$ will converge to the optimal $Q^*(s, a)$ w.p.1. Theorem1 proves that the critic network (Q network) is guaranteed to approximate the optimal value function. For the convergence of the actor network in the CDTA request net, since the gradient update direction of the actor network is determined by the critic network, as long as the critic network can be guaranteed to converge to the optimal action-value function, we can obtain an approximate optimal request policy.

TABLE I
UAVs' COMBAT CAPABILITY

Type(i)	att(U^i)	rec(U^i)	jam(U^i)	com(U^i)	bom(U^i)	MFC
1	30	3	4	5	5	40
2	2	25	4	5	3	45
3	7	4	40	3	2	50
4	2	3	4	36	6	48
5	6	3	5	7	42	55

E. Reinforcement Learning Objective

With the element of the MDP defined for the request and response UAV, it is clear that the dynamic task allocation is converted to learn the optimal request and response policy π to solve the reinforcement learning in (4). Here, N refers to the total number of dynamic tasks in the mission.

IV. The Principle of Distributed Task Allocation

In the multi-agent system, the UAV is classified by its functions and behaviors. The UAV that discovers the new task is named $Proposer$, and the UAVs that receive the request information are named $Responser$, in which agree to the request and ultimately participate in the new task are called *Participants*. Before the task is initialized, we have no prior knowledge of the role information of these UAVs. After initializing the task, we randomly generate new task requirement information and randomly set the state of all UAVs. Each UAV state can be one of (*idle, busy, committed*). If the UAV is the $Proposer$ or is performing a task, it's state is *busy*. If the UAV is not currently serving a task or the task has been executed, the state is *idle*. And if the $Responser$ participates in a new task, the state is updated to *committed*. In the multi-agent system, after a new task appears, the $Proposer$ makes an online decision on the optimal $Responsers$ through the coordinated dynamic task allocation request network. The $Responser$s who receive the task information uses the DQN-based task evaluation network to evaluate the expected reward of participating in the new task. If the expected reward of participating in the new task is higher than the current state, the $Responser$ sends an acceptance message, otherwise sends a rejection message to the $Proposer$. Finally, the $Proposer$ determines the UAVs sequence for task reallocation according to the task requirement and considering the distance between the UAV and the task target in (1) and (2). The dynamic task reallocation strategy is learned based on it's past allocation experience. The pseudocode of the algorithm is shown in Algorithm 1. The idea of the algorithm is illustrated as follows:

- When the UAV denoted as $Proposer$ finds a new task, it broadcast the task announcement to the $Responsers$. It uses CDTA request network to determine the optimal request UAVs. The CDTA request network takes the state

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE MAIN HYPERPARAMETERS OF THE CDTA

Hyperparameter	Memory	learning rate	Batch size	ε	γ	α	β
Value	10^5	0.001	64	0.9	0.99	0.92	0.85

of all the agent and task resources as input and outputs actions for all UAVs. If the UAV's action is true, the UAV is named $Responser$, and sent task announcement. We have the $ResponserList = \{U_1, U_2, \ldots U_k\}$ and the message sending to the $Responsers, TaskResourceAnnounce = \{Resource(T_new)\}$.

- These UAVs in the $ResponserList$ receiving the $TaskResourceAnnounce$ message sent by the $Proposer$.
- If (state($Responser$)=*idle*) Then the $Responser$ UAVs send the types and the capability it contains, the distance cost and the Q value namely $ResponserMessage = \{UAV(ID), Capability(U_k), Distance(U_k), Q(U_k)\}$.
- Else(state($Responser$)=*busy*) The $Responser$ does not directly refuses but calculate the expected reward (Q value) of the current task and the new task with the Q network. $ResponserMessage = \{UAV(ID), Capability(U_k), Distance(U_k), Q_Now(U_k), Q_New(U_k)\}$.

The $Proposer$ receives the $ResponserMessage$.

- If(the new task source requirement is satisfied with the UAVs that are in $idle$ state) Then the $Proposer$ will select the UAVs with higher Q value to participate in the task under the constraints of (1) and (2).
- Else: According to (1) and (2), it will select corresponding UAVs with higher Q-value on the new task, then recalculate whether the original task meets the requirements.

The $Proposer$ sends a contract to the selected UAVs, which will participate in the new task. The state of the selected UAVs will be changed to *committed*. $Contract = \{UAV(ID), TaskID(U_k), Capability(U_k)\}$.

- After recalculating the original task requirement, whether there are new tasks appears is classified. This process will continue until the requirement of the new tasks is satisfied. We get the UAVs set for reallocation, $UAV_Reallocate = \{U_1, U_2, \ldots U_i\}$. The $Proposer$ gives priority to selecting UAVs with a small distance cost.

V. EXPERIMENTS

In order to verify the convergence performance and effectiveness of the proposed approach, experiment simulations are divided into two parts in this section. In the simulation result for setting1, the random experience replay is compared with prioritized experience replay method and the prioritized experience replay is proven to converge faster. In Simulation result for setting2, the scalability of the algorithm for a variety of number of UAVs is tested and the approach is compared with the fast task allocation method in [22] and coalitional game task allocation method in [33]. Furthermore, the fitness value is adopted to evaluate the dynamic allocation with the changing number of UAVs.

A. Experimental Settings

In this section the experiment setting for the task allocation scenario is introduced. A dynamic task generation environment based on Python is designed, which can generate initial task allocation results and UAV resources, and randomly generate dynamic task requirements. It is guaranteed that the UAV resources are redundant, that is, the UAV resources can satisfy the old characters and the new tasks at the same time. The input of this environment is the index of the UAV performing dynamic tasks, and the output is the task index of the remaining tasks and the state of all UAVs. Suppose 5 types of UAVs have been initially allocated for different tasks. The combat capability of each type of UAV is given in Table I, where MFC means the maximum fuel consumption. The distance of the task performed by the UAV can not exceed the MFC. The MFC of the new task ranges in [30, 50]. We also set the ranges of attack capability, reconnaissance capability, jamming capability, communication and bombing capability for the new task as [30, 390], [25, 325], [40, 520], [36, 468], [42, 546]. When the new task appears in combat, some UAV may have completed their tasks, or may not. To simulate the effect of environment uncertainty, we randomly set the state of the UAVs (10% of the UAVs are in *idle* state).

To evaluate the performance of the task allocation strategy, the threshold defined in (3) is set to 45, which means that only one additional UAV is allowable at most.The hyperparameters are illustrated in Table II. The CDTA request net uses a bidirectional LSTM network and a fully connected network with eight hidden layers. The activation function used for the network is rectified linear unit(Relu). The DQN network uses a fully connected network with three hidden layers. The algorithm is trained on a computer with Intel i5-8500 and 24 GB of memory.

B. Simulation Results for Setting1

Prioritized experience replay is employed on CDTA and compared with random experience replay. We use the identical neural network architecture, optimizer, learning algorithm, experience replay buffer size, and the random seed of the environment. The only difference of the algorithm is the experience sampling method. The prioritized sampling method introduces new hyperparameters. We compare different combination of α and β, and find the sweet spot to be $\alpha = 0.92$, $\beta = 0.85$. We collect the average Q values of the critic network from a random policy and track the change of the Q value in training. In Fig. 4, it can be seen that the average Q value per episode gradually gets higher with the increase of the training episodes. The prioritized sampling method learning curve converges faster than the random sampling method. Intuitively, prioritized experience replay prefers to extract a more valuable experience according to TD-error to accelerate the training process.

To verify the efficiency of the trained CDTA request network and the DQN-based evaluation network, we set up a random task, which requires five different types of 10–180 UAVs. Fig. 5 shows the allocation result of the CDTA algorithm, where the

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Coordinated Dynamic Task Allocation Algorithm.

for episode $= 1$ to M (Max episodes) **do**
  Initialize the random state of the UAV and the requirement of the dynamic task and the performed task.
  Classify the roles of the UAVs and obtain the initial state of the UAVs.
  Receive the initial observation s_p, s_r of the $Propser$ and $Responser$
  for $t = 1$ to N (N dynamic tasks) **do**
    For the $Proposer$ UAV, choose action $a_p^t = \pi_i(Require_Task_new, Task_UAV)$ for the current policy, according to the state s_p^t.
    According to the $ResponserList$, the $Proposer$ sends the $TaskResourceAnnounce$ to the $Responsers$.
    The $Responsers$ UAVs in the $ResponserList$, receive the $TaskResourceAnnounce$ message.
    if state($Responser$)=$idle$ **then**
      The $Responser$ send the $ResponserMessage = \{UAV(ID), Capability(U_k), Distance(U_k), Q(U_k)\}$, according to the $Responser$ state s_r^t.
    else
      The $Responser$ send the $ResponserMessage = \{UAV(ID), Capability(U_k), Distance(U_k), Q_Now(U_k), Q_New(U_k)\}$, according to the $Responser$ state s_r^t.
    end if
    The $Proposer$ receives the $ResponserMessage$, and select the UAVs with higher Q value to participate in the task under the constraints of (1) and (2).
    The $Proposer$ sends a contract to the selected UAVs. The state of the selected UAVs will be changed to $commited$.
    Receive reward of the $Proposer$ and $Responser$: R_p, R_r and observe new state: s_p^{t+1}, s_r^{t+1}
    Store $(s_p^t, a_p^t, R_p, s_p^{t+1})$, $(s_r^t, a_r^t, R_r, s_r^{t+1})$ in replay buffer $\mathcal{R}$
    $s_p^t \leftarrow s_p^{t+1}$, $s_r^t \leftarrow s_r^{t+1}$
    for agent $i = 1$ to Number of agents (N) **do**
      if agent=$Proposer$ **then**
        Sample a prioritized minibatch of M samples $(s_p^t, a_p^t, R_p, s_p^{t+1})$
        The $Responser$ send the $ResponserMessage = \{UAV(ID), Capability(U_k), Distance(U_k), Q(U_k)\}$, according to the $Responser$ state s_r^t.
        Set $y = r + \gamma Q_i^{\pi'}(s_p, a_1', .., a_N')|_{a'_j = \pi'_j(s_p)}$
        compute critic gradient estimation according to 14:
        $\nabla_\varphi L(\varphi) = \nabla_\varphi \frac{1}{M} \sum_{m=1}^{M} E_{s_p,a,r,s'_p}[w_i \cdot (Q_i^\pi(s_p, a_1, \ldots, a_N) - y)^2]$
        compute actor gradient estimation according to 10:
        $\nabla_\theta J(\pi) = E_{s_p,a\sim D}[\nabla_\theta \pi_i(a_i|s_p) \nabla_{\pi_i} Q_i^\pi(s_p, a_1, \ldots, a_N|a_i = \pi_i(s_p))]$
      else
        Sample a random minibatch of J samples $(s_r^t, a_r^t, R_r, s_r^{t+1})$.
        Update the Deep Q-Learning Network by minimizing the loss 18:
        $loss = E[(r + \gamma \max_{a'} Q(s_r', a_r') - Q(s_r, a_r))^2]$
      end if
    end for
    Update target network parameters for each agent i:
    $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$, $\varphi' \leftarrow \tau\varphi + (1-\tau)\varphi'$
  end for
end for

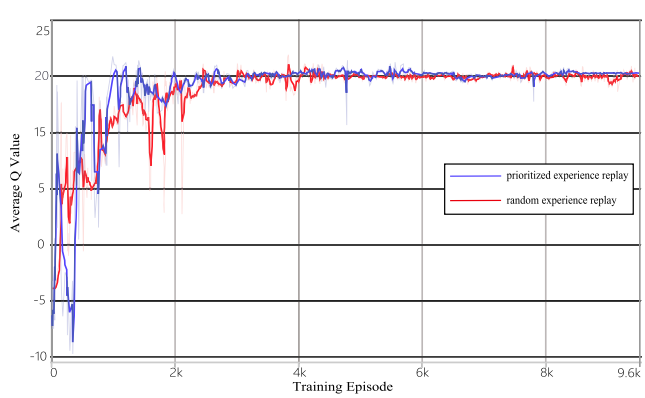

Fig. 4. The average Q value of the priority experience replay method and random experience replay per episode.

X-axis denotes the number of UAVs and Y-axis represents the cumulative reward of each allocation. As defined in (7), a reward of -0.5 is obtained each time a UAV sends a request, and a reward of 30 is obtained immediately when the request UAV sends the request to the proper UAV. Once the requesting UAV sends a wrong request to a UAV, a reward of -10 will be obtained. If the agent chooses the action with the most utility, a reward of 2 is obtained immediately; otherwise, it will get a reward of -2. If the selected UAVs that are sent the contact meet the (2) requirement, a reward of 50 is obtained. Due to the distributed response mechanism, the number of responding UAVs increases as the number of UAVs increases, so an averaging is used to calculate responding UAVs' reward calculation, which means the total cumulative reward obtained ranges from 50 to 82 if the reallocation result is valid.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

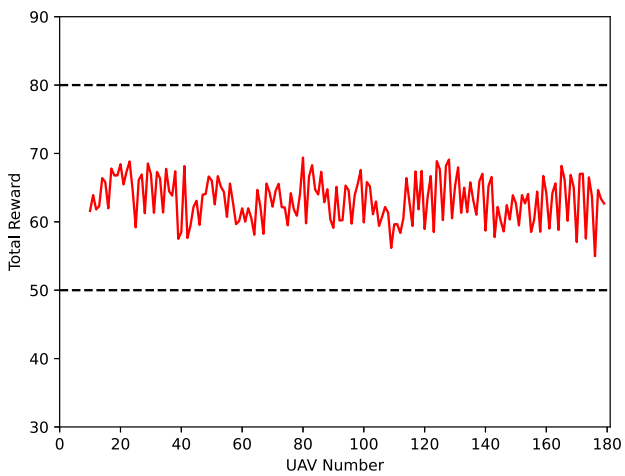

Fig. 5. The cumulative reward.

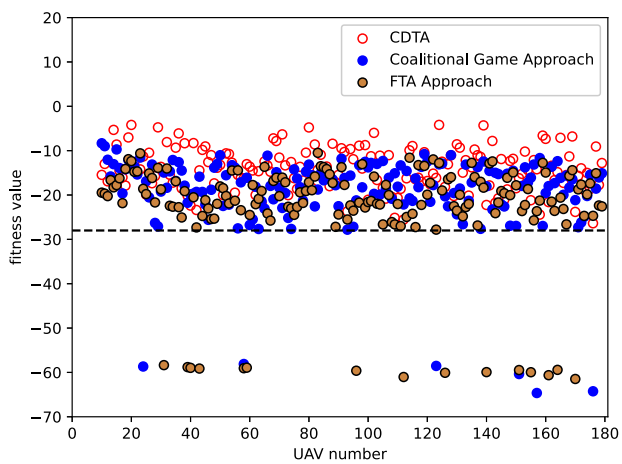

Fig. 6. The fitness value.

C. Simulation Results for Setting2

To illustrate the advantage of our proposed CDTA algorithm, comparative simulations are verified in this section with the coalitional game and the fast task allocation approach. N dynamic tasks are randomly generated to evaluate the result of the task assignment. The fitness value is defined based on the constraints and evaluation function used to assess the three algorithms performance (allocation result) in Fig. 6. Consider the constraints in (2) and the evaluation function (3), fitness value is defined as follows:

$$\max Fitness = -\frac{1}{N}\sum_{k=1}^{N} w_k J_k - p_1 \sum_{k=1}^{N} pe_k^{att} - p_2 \sum_{k=1}^{N} pe_k^{rec} - p_3 \sum_{k=1}^{N} pe_k^{jam} - p_4 \sum_{k=1}^{N} pe_k^{com} - p_5 \sum_{k=1}^{N} pe_k^{bom}, \tag{34}$$

where J_k represents kth the evaluation function of dynamic task and $Pe_k^{att}, Pe_k^{rec}, Pe_k^{jam}, Pe_k^{com}, Pe_k^{bom}$ represents the penalty function which is defined in (35).

If the allocated UAVs meet the requirement of the specific dynamic task, then the corresponding penalty function will be 0, otherwise will be 1. p_1, p_2, p_3, p_4, p_5 are the weight of the penalty function corresponding to the dynamic task. w_k is the importance of the dynamic task. In this case, $p_i (i = 1, \ldots, 5)$ is given as $\{40, 40, 40, 40, 40\}$ and w_k of the new task is given as $\{3, 2.8, 2.6, 2.4, 2.2, 2, 1.8, 1.6, 1.4, 1.2\}$. N represents the number of dynamic tasks. To simulate the various environment of the battlefield, we randomly generate 10 dynamic tasks for the number of UAV varying from 10 to 180. Fig. 6 gives the fitness value with the changing number of UAVs. It can be observed that the coalitional game and the FTA approach have a lower effect than the CDTA algorithm. From the simulation results above, we can conclude that CDTA demonstrates better allocation performance than the coalitional game and FTA in the presence of changing number of UAVs.

$$\begin{aligned}
Pe_j^{att} &= \begin{cases} 1 & \text{if} \quad att\left(T_j\right) > \sum_{i=1}^{M} att\left(U_i^j\right) \\ 0 & \text{else,} \end{cases} \\
Pe_j^{rec} &= \begin{cases} 1 & \text{if} \quad rec\left(T_j\right) > \sum_{i=1}^{M} rec\left(U_i^j\right) \\ 0 & \text{else,} \end{cases} \\
Pe_j^{jam} &= \begin{cases} 1 & \text{if} \quad jam\left(T_j\right) > \sum_{i=1}^{M} jam\left(U_i^j\right) \\ 0 & \text{else,} \end{cases} \\
Pe_j^{com} &= \begin{cases} 1 & \text{if} \quad com\left(T_j\right) > \sum_{i=1}^{M} com\left(U_i^j\right) \\ 0 & \text{else,} \end{cases} \\
Pe_j^{bom} &= \begin{cases} 1 & \text{if} \quad bom\left(T_j\right) > \sum_{i=1}^{M} bom\left(U_i^j\right) \\ 0 & \text{else.} \end{cases}
\end{aligned} \tag{35}$$

TABLE III
THE RUNNING TIME OF THE CDTA AND COALITIONAL GAME APPROACH

Methods	N=50	N=70	N=100	N=120	N=180
CDTA(s)	0.0181452	0.0213623	0.0296553	0.0339776	0.0397077
CG(s)	0.037992768	0.090718168	0.232519716	0.33208989	0.81732876
FTA(s)	0.0086067	0.0109037	0.0157421	0.0208751	0.0373506

In the scenario described above, the realtime performance in the scalability of the three algorithms are illustrated. As the Table III shows, the CDTA and the FTA which are designed based on the neural network have higher realtime advantages. The running time of the CDTA algorithm increases linearly with the number of UAVs, while the coalitional game algorithm increases exponentially. Due to the simple network structure, the FTA algorithm has great advantages in real-time performance, but the quality of the solution cannot be guaranteed. With the same number of UAVs, while ensuring the effectiveness of the solution, the CDTA algorithm has more advantages in real-time.

VI. CONCLUSION

This paper studied the dynamic task allocation problem of heterogeneous UAVs in the presence of uncertain environments. The dynamic task allocation concerned in this paper is introduced with constraints and evaluation function. Then the task

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

allocation problems are formulated as a multi-agent stochastic game, turning it into an MDP problem. The $Proposer$ and $Responser$ were defined in the distributed dynamic task allocation, and a multi-agent reinforcement learning algorithm is constructed for the $Proposer$ to send requests and a Q-learning-based algorithm for the $Responser$ to decide whether to accept the request. The proposed algorithm has a better realtime performance, high scalability for a lager group of UAVs. Comparison simulations with the coalitional game and FTA approach show that the proposed algorithm can solve the dynamic task allocation problem more fast and efficiently.

REFERENCES

[1] R. R. Murphy, *Disaster Robotics*. Cambridge, MA, USA: MIT Press, 2014.

[2] J. Parker, E. Nunes, J. Godoy, and M. Gini, "Exploiting spatial locality and heterogeneity of agents for search and rescue teamwork," *J. Field Robot.*, vol. 33, no. 7, pp. 877–900, 2016.

[3] Y. Liu and G. Nejat, "Robotic urban search and rescue: A survey from the control perspective," *J. Intell. Robotic Syst.*, vol. 72, no. 2, pp. 147–165, 2013.

[4] A. Quattrini Li, R. Cipolleschi, M. Giusto, and F. Amigoni, "A semantically-informed multirobot system for exploration of relevant areas in search and rescue settings," *Auton. Robots*, vol. 40, no. 4, pp. 581–597, 2016.

[5] Y.-J. Chen, D.-K. Chang, and C. Zhang, "Autonomous tracking using a swarm of UAVs: A constrained multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13702–13717, Nov. 2020.

[6] A. Whitbrook, Q. Meng, and P. W. Chung, "A novel distributed scheduling algorithm for time-critical multi-agent systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2015, pp. 6451–6458.

[7] J. Turner, Q. Meng, and G. Schaefer, "Increasing allocated tasks with a time minimization algorithm for a search and rescue scenario," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 3401–3407.

[8] Y. Zhang, Z. Mou, F. Gao, J. Jiang, R. Ding, and Z. Han, "UAV-enabled secure communications by multi-agent deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 11599–11 611, Oct. 2020.

[9] A. Gao, Q. Wang, W. Liang, and Z. Ding, "Game combined multi-agent reinforcement learning approach for UAV assisted offloading," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 12888–12901, Dec. 2021.

[10] H. Huang and A. V. Savkin, "Navigating UAVs for optimal monitoring of groups of moving pedestrians or vehicles," *IEEE Trans. Veh. Technol.*, vol. 70, no. 4, pp. 3891–3896, Apr. 2021.

[11] H. Huang, Y. Yang, H. Wang, Z. Ding, H. Sari, and F. Adachi, "Deep reinforcement learning for UAV navigation through massive MIMO technique," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 1117–1121, Jan. 2020.

[12] H. Qin et al., "Autonomous exploration and mapping system using heterogeneous UAVs and UGVs in GPS-denied environments," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1339–1350, Feb. 2019.

[13] Z. Qin, Z. Liu, G. Han, C. Lin, L. Guo, and L. Xie, "Distributed UAV-BSs trajectory optimization for user-level fair communication service with multi-agent deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 12290–12301, Dec. 2021.

[14] K.-S. Goetzmann, S. Stiller, and C. Telha, "Optimization over integers with robustness in cost and few constraints," in *Proc. Int. Workshop Approximation Online Algorithms*, 2011, pp. 89–101.

[15] S. S. Ponda, L. B. Johnson, A. N. Kopeikin, H.-L. Choi, and J. P. How, "Distributed planning strategies to ensure network connectivity for dynamic heterogeneous teams," *IEEE J. Sel. Areas Commun.*, vol. 30, no. 5, pp. 861–869, Jun. 2012.

[16] C. Sun, X. Wang, H. Qiu, and Q. Zhou, "Game theoretic self-organization in multi-satellite distributed task allocation," *Aerosp. Sci. Technol.*, vol. 112, 2021, Art. no. 106650.

[17] H. T. Ozdemir and C. K. Mohan, "Evolving schedule graphs for the vehicle routing problem with time windows," in *Proc. Congr. on Evol. Computation*, 2000, pp. 888–895.

[18] T. Takeno, Y. Tsujimura, and G. Yamazaki, "A single-phase method based on evolution calculation for vehicle routing problem," in *Proc. 4th Int. Conf. Comput. Intell. Multimedia Appl.*, 2001, pp. 103–107.

[19] K. Uchimura, H. Sakaguchi, and T. Nakashima, "Genetic algorithms for vehicle routing problem in delivery system," in *Proc. Veh. Navigation Inf. Syst. Conf.*, 1994, pp. 287–290.

[20] R. Montemanni, L. M. Gambardella, A. E. Rizzoli, and A. V. Donati, "Ant colony system for a dynamic vehicle routing problem," *J. Combinatorial Optim.*, vol. 10, no. 4, pp. 327–343, 2005.

[21] Y. Jin, Y. Liao, A. A. Minai, and M. M. Polycarpou, "Balancing search and target response in cooperative unmanned aerial vehicle (UAV) teams," *IEEE Trans. Syst., Man, Cybern., Part B (Cybern.)*, vol. 36, no. 3, pp. 571–587, Jun. 2006.

[22] X. Zhao, Q. Zong, B. Tian, B. Zhang, and M. You, "Fast task allocation for heterogeneous unmanned aerial vehicles through reinforcement learning," *Aerosp. Sci. Technol.*, vol. 92, pp. 588–594, 2019.

[23] S. Ichoua, M. Gendreau, and J.-Y. Potvin, "Planned route optimization for real-time vehicle routing," *Dynamic Fleet Manage.*, vol. 38, pp. 1–18, 2007.

[24] J. I. v. Hemert and J. A. La Poutré, "Dynamic routing problems with fruitful regions: Models and evolutionary computation," in *Proc. Int. Conf. Parallel Problem Solving Nature*, 2004, pp. 692–701.

[25] N. Jodeh, M. Mears, and D. Gross, "An overview of the cooperative operations in urban terrain (counter) program," in *Proc. AIAA Guid., Navigation Control Conf. Exhibit*, 2008, Art. no. 6308.

[26] D. Liu et al., "Task-driven relay assignment in distributed UAV communication networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11003–11017, Nov. 2019.

[27] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, Aug. 2009.

[28] N. Buckman, H.-L. Choi, and J. P. How, "Partial replanning for decentralized dynamic task allocation," in *Proc. AIAA Scitech 2019 Forum*, 2019, Art. no. 0915.

[29] J. Turner, Q. Meng, G. Schaefer, A. Whitbrook, and A. Soltoggio, "Distributed task rescheduling with time constraints for the optimization of total task allocations in a multirobot system," *IEEE Trans. Cybern.*, vol. 48, no. 9, pp. 2583–2597, Sep. 2018.

[30] C. Chen, W. Bao, T. Men, X. Zhu, J. Wang, and R. Wang, "Nectar-an agent-based dynamic task allocation algorithm in the UAV swarm," *Complexity*, vol. 2020, pp. 1–4, 2020.

[31] X. Wang and B. Sheng, "Multi-robot task allocation algorithm based on anxiety model and modified contract network protocol," in *Proc. IEEE 2nd Inf. Technol., Netw., Electron. Automat. Control Conf.*, 2017, pp. 1606–1612.

[32] H. Wu and H. Shang, "Potential game for dynamic task allocation in multi-agent system," *ISA Trans.*, vol. 102, pp. 208–220, 2020.

[33] I. Jang, H.-S. Shin, and A. Tsourdos, "Anonymous hedonic game for task allocation in a large-scale multiple agent system," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1534–1548, Dec. 2018.

[34] V. Mnih et al., "Playing Atari with deep reinforcement learning," in *Proc. NIPS Deep Learn. Workshop*, 2013. [Online]. Available: http://arxiv.org/abs/1312.5602

[35] J. Hu, H. Zhang, and L. Song, "Reinforcement learning for decentralized trajectory design in cellular UAV networks with sense-and-send protocol," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6177–6189, Aug. 2019.

[36] J. Hu, H. Zhang, L. Song, Z. Han, and H. V. Poor, "Reinforcement learning for a cellular internet of UAVs: Protocol design, trajectory control, and resource management," *IEEE Wireless Commun.*, vol. 27, no. 1, pp. 116–123, Feb. 2020.

[37] J. Hu, H. Zhang, L. Song, R. Schober, and H. V. Poor, "Cooperative internet of UAVs: Distributed trajectory design by multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 6807–6821, Nov. 2020.

[38] D. Silver et al., "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[39] Y. Liu, H. Liu, Y. Tian, and C. Sun, "Reinforcement learning based two-level control framework of UAV swarm for cooperative persistent surveillance in an unknown urban area," *Aerosp. Sci. Technol.*, vol. 98, 2020, Art. no. 105671.

[40] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[41] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 285–292.

[42] P. Peng et al., "Multiagent bidirectionally-coordinated nets: Emergence of human-level coordination in learning to play starcraft combat games," 2017, *arXiv:1703.10069*.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.

[43] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. Int. Conf. Learn. Representation*, 2016. [Online]. Available: http://arxiv.org/abs/1511.05952
[44] S. Zhang et al., "Architectural complexity measures of recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., 2016, pp. 1830–1838. [Online]. Available: https://proceedings.neurips.cc/paper/2016/file/860320be12a1c050cd7731794e231bd3-Paper.pdf
[45] M. Sipper, "A serial complexity measure of neural networks," in *Proc. IEEE Int. Conf. Neural Netw.*, 1993, pp. 962–966.
[46] T. Jaakkola, M. Jordan, and S. Singh, "Convergence of stochastic iterative dynamic programming algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 1993, vol. 6, pp. 703–710.

Da Liu received the B.S. degree in electrical engineering and automation from the Civil Aviation University of China, Tianjin, China. He is currently working toward the Ph.D. degree with the Department of Electrical and Information Engineering, Tianjin University, Tianjin, China. His research interests include multiagent reinforcement learning, multiagent scheduling and allocation, and multi-UAV prediction and confrontation.

Liqian Dou received the B.S., M.S., and Ph.D. degrees in control science and engineering from Tianjin University, Tianjin, China, in 1999, 2005, and 2008, respectively. From 2015 to 2016, he was an Academic Visitor with the School of Electrical and Electronic Engineering, University of Manchester, Manchester, U.K.He is currently an Associate Professor with the Department of Control Science and Engineering, School of Electrical and information Engineering, Tianjin University. His research interests include nonlinear control for hypersonic vehicle, attitude control of spacecraft, and fault tolerance for spacecraft.

Ruilong Zhang received the M.S. and Ph.D. degrees in control science and engineering from Tianjin University, Tianjin, China. He is currently an Engineer with the Beijing Aerospace Automatic Control Institute, Beijing, China. His research interests include multiagent reinforcement learning and multiUAV prediction.

Xiuyun Zhang received the B.S. degree in automation from Qingdao University, Qingdao, China, in 2014, and the M.S. and Ph.D. degrees in automatic control from Tianjin University, Tianjin, China, in 2016 and 2020, respectively. She is currently with the School of Electrical and Information Engineering, Tianjin University. Her main research interests include fault diagnosis and fault tolerant control for spacecraft, finite-time multiagent formation control and guidance, and control and simulation for flight vehicle.

Qun Zong is currently a Professor and one of the academic pacesetters in control theory and control engineering with the School of Electrical and Information Engineering, Tianjin University, Tianjin, China. He is also the Director of the New Aircraft Guidance and Control Center, Ministry of Education, the Expert Groups Deputy Head of the Major Project of the Chinese Ministry of Education, Beijing, China, and the Deputy Director of the New Aircraft Joint Research Center, Tianjin. His research interests include guidance, control and simulation for flight vehicles, coordination control of multiagent systems, fault diagnosis and fault-tolerant control, complex system modeling, and optimization control.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 07,2026 at 12:17:06 UTC from IEEE Xplore. Restrictions apply.